

UFME7R-15-M Robot Learning for Control

Written Report Template: Develop a Method for Planar Robot Manipulation

```
clearvars; clear; clc;
```

How do I use this file?

This MLX file serves as the template for your main coursework submission. Please read the assessment brief on Blackboard and complete this MLX file along with the class files SkillGeneralisation.m, MixtureGaussian.m, and the Simulink project RobotSimulation.slx to produce a complete submission. To ensure the code in this MLX runs correctly, you must complete the required functions within SkillGeneralisation.m, MixtureGaussian.m, and RobotSimulation.slx. Please read the instructions carefully before starting your work.

Your final submission should include a PDF export of your MLX file, as well as a compressed ZIP file containing your complete working workspace, including the finished MLX file, all your code, and any data used to support your written report. Please note that the final marking will be based on the PDF only, so make sure that everything required is fully included in the PDF document. If you are unsure how to use an MLX file or how to export it for submission, please refer to the guide available on Blackboard Ultra.

If you are comfortable writing MATLAB functions and wish to use them to organise your code, please define them as static functions/methods inside SkillGeneralisation.m rather than creating additional standalone files. If this is not possible for any reason, you may include the additional files in your submission.

Please also note the following:

1. Ensure you are using a MATLAB version of R2025b or newer, and that both Simulink and Simscape are installed.
2. There are some outputs in the template, which are used as reference. Please delete them before you start to add your content.

3. Your PDF exported from this MLX will be marked and checked with your completed class files. Your MATLAB files are used to support what you have documented in the PDF, but will not be marked themselves.
4. Very wide images do not render well onto the PDF. Please check to make sure images you insert are captured appropriately within the PDF.

Task 1: Introduction (5 marks)

1. Utilise Dynamic Movement Primitives (DMPs) alongside Gaussian Mixture Models (GMM) and Gaussian Mixture Regression (GMR) to devise an enhanced Dynamic Movement Primitives for single-arm robot manipulation.
2. During the development of the new algorithm, you can refer to the paper "DMP and GMR based Teaching by Demonstration for a KUKA LBR Robot" by Hewitt et al., introduced in the lecture.
3. Write an introduction about the calculation steps based on your understanding of this paper, into your own report.
4. Please ensure that the algorithms you introduce are consistent with the robot-learning implementation in Task 2.

Useful materials for this Part include:

1. Lecture notes on DMPs
2. Lectures on Linear model, GMM and GMR
3. Describe your calculations below (max 200 words):

Describe your calculations below (max 200 words):

A DMP, as I understood it, is built from two coupled systems. The canonical system plays the role of an internal clock. It produces a phase variable s that decays monotonically from 1 to 0:

$$\tau \dot{s} = -\alpha_s s$$

The transformation system is then a spring-damper attractor pulling the Cartesian state x towards the goal g , modulated by a forcing term that shapes the trajectory:

$$\tau \dot{\nu} = \beta(g - x) - \alpha \nu + s f(s), \quad \tau \dot{x} = \nu$$

From there, I broke the calculation into four steps.

First, I extract target forcing values from each demonstration by rearranging the transformation system.

Second, in the original DMP these values are fitted by LWR over fixed-centre Gaussian basis functions — but as the paper points out, this only works for a single demonstration and forces me to place basis centres manually.

Third, the enhanced version instead encodes the joint distribution $P(s, f)$ as a GMM, which I initialise with K-means and train with Expectation-Maximisation.

Fourth, I apply GMR to recover the conditional expectation:

$$\hat{f}(s) = \sum_{k=1}^K \beta_k [\mu_{f,k} + \Sigma_{fs,k} \Sigma_{s,k}^{-1} (s - \mu_{s,k})]$$

GMR naturally fuses multiple demonstrations, learns component centres automatically, captures trajectory variance, and provides a prediction cost independent of dataset size.

Task 2: Robot Learning (20 marks total)

Implement your improved DMP method using the following steps:

Part 1: Original DMPs (3 marks)

1. Employ the original DMP to compute the values of nonlinear term (the forcing term) of the inputs.
2. Generate more plots to demonstrate the trajectories reconstructed by the DMPs with different parameters.

Useful materials for this Part include:

1. Lecture notes on DMPs
2. Exercise for DMPs

3. Documentation for loading data into MATLAB: <https://uk.mathworks.com/help/matlab/ref/load.html>

The below code provides a starting point for Part 1. You need to complete the functions in the SkillGeneralisation.m to make the code work.

$$\gamma_i = \frac{\sum_t \psi_i(t) \cdot s(t)^2 \cdot f_{\text{target}}(t)}{\sum_t \psi_i(t) \cdot s(t)^2}$$

$$f(s) = \sum_{i=1}^9 \gamma_i \phi_i(s)$$

```
nbData = 200;           % Length of each demo trajectory
nbDemos = 5;            % Number of demonstrations for the DMP training
```

Define DMP parameters

```
nStates = 9;           % Number of activation functions (i.e., number
of states in the forcing term)
nVar = 3;              % Number of variables [s,f1,f2] (decay term and
perturbing force)
beta = 50;             % Stiffness gain (β)
alpha = (2*beta)^.3;   % Damping gain (with ideal underdamped damping
ratio) (α)
decayFactor = 1.1;     % Decay factor (a in τx'=-ax)
dt = 0.01;            % Duration of time step (τ)
```

Load handwriting trajectories

```
demos = [];
load('Demonstrations.mat');
demos_in = {demos{1:nbDemos}};
```

Initialise the DMP instance

```
dmp = SkillGeneralisation(nVar, nStates, beta, alpha, dt); % Class
constructor
dmp = dmp.canonicalSystemInitialisation(decayFactor, nbData);
dmp = dmp.trajectory2Forcing(demos_in);
fWLR = dmp.fittingWithLocallyWLS();
trajWLR = dmp.forcing2Trajectory(fWLR);
```

```
disp(size(fWLR))           % expected (2, 200)
```

```
2    200
```

```
disp(fWLR(:, 1:5))        % first 5 columns
```

```
155.1926  153.5924  150.9822  146.7889  140.1921
474.2346  474.2090  474.1671  474.0994  473.9917
```

```
disp(size(trajWLR))       % expected (2, 200)
```

```
2    200
```

```
disp(trajWLR(:, 1:10))
```

```
7.7731    7.7279    7.6613    7.5737    7.4657    7.3376    7.1892    7.0200    6.8294    6.6161
7.7587    7.7702    7.7862    7.8060    7.8288    7.8538    7.8804    7.9078    7.9353    7.9622
```

```
disp(demos_in{2}.pos(:, 1:10))
```

```
7.9781    7.9783    7.9776    7.9703    7.9177    7.8741    7.8095    7.7265    7.6312    7.4896
8.3798    8.3798    8.3797    8.3781    8.3704    8.3666    8.3627    8.3588    8.3550    8.3512
```

```
disp(['MAX: ' num2str(max(abs(trajWLR(:))))]) % > 100 indicates divergence
```

```
MAX: 8.0792
```

$$f_{target} = \frac{\ddot{x} - \beta(g - x) + \alpha \dot{x}}{s}$$

Regression with Locally Weighted Linear Regression (WLR)

```
fWLR = dmp.fittingWithLocallyWLS();
trajWLR = dmp.forcing2Trajectory(fWLR);
```

- fWLR

$$\gamma_i = \frac{\sum_t \psi_i(t) \cdot s(t)^2 \cdot f_{target}(t)}{\sum_t \psi_i(t) \cdot s(t)^2}$$

$$f(s) = \sum_{i=1}^9 \gamma_i \phi_i(s)$$

```
disp(size(fWLR))          % (2, 200) 이어야 함
```

```
2    200
```

```
disp(fWLR(:, 1:5))        % 처음 5개 값
```

```
155.1926 153.5924 150.9822 146.7889 140.1921
474.2346 474.2090 474.1671 474.0994 473.9917
```

```
disp(size(trajWLR))           % (2, 200) 이어야 함
```

```
2    200
```

```
disp(trajWLR(:, 1:10))
```

```
7.7731 7.7279 7.6613 7.5737 7.4657 7.3376 7.1892 7.0200 6.8294 6.6161
7.7587 7.7702 7.7862 7.8060 7.8288 7.8538 7.8804 7.9078 7.9353 7.9622
```

```
disp(demos_in{2}.pos(:, 1:10))
```

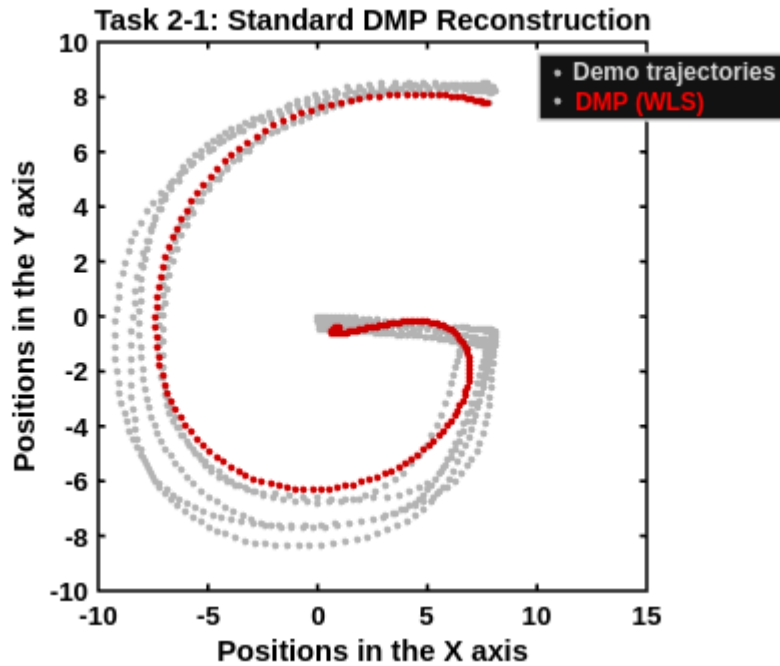
```
7.9781 7.9783 7.9776 7.9703 7.9177 7.8741 7.8095 7.7265 7.6312 7.4896
8.3798 8.3798 8.3797 8.3781 8.3704 8.3666 8.3627 8.3588 8.3550 8.3512
```

```
disp(['MAX: ' num2str(max(abs(trajWLR(:))))]) % 100 넘으면 발산
```

```
MAX: 8.0792
```

Spatial plot

```
% Axis-color safety net (re-applies global styling in case this cell is run
alone)
set(groot, 'DefaultFigureColor','w', 'DefaultAxesColor','w', ...
'DefaultAxesXColor','k', 'DefaultAxesYColor','k',
'DefaultAxesZColor','k', ...
'DefaultAxesGridColor',[0.7 0.7 0.7], ...
'DefaultAxesLineWidth',1.2, 'DefaultAxesFontWeight','bold', ...
'DefaultAxesBox','on', 'DefaultTextColor','k');
try, set(groot, 'defaultFigureTheme', 'light'); end %#ok<TRYNC>
figure('Color', 'w');
for n = 1:nbDemos
    plot(demos_in{n}.pos(1,:), demos_in{n}.pos(2,:), ...
        'o', 'markersize', 8, 'color', [.7 .7 .7]);
    hold on
end
plot(trajWLR(1,:), trajWLR(2,:), 'o', 'markersize', 8, 'color', [.8 0 0]);
xlabel('Positions in the X axis')
ylabel('Positions in the Y axis')
xlim([-10 15])
ylim([-10 10])
axis square
legend({'Demo trajectories', '\color{red}DMP (WLS)'}, 'Location', 'best')
title('Task 2-1: Standard DMP Reconstruction')
ax=gca; set(ax, 'Color','w', 'XColor','k', 'YColor','k', 'GridColor',[0.7 0.7
0.7]); ax_.Title.Color='k'; ax_.XLabel.Color='k'; ax_.YLabel.Color='k';
hold off
```



Comparison of DMP Learning Results for Different Numbers of Basis Functions (K)

```
% Axis-color safety net (re-applies global styling in case this cell is run
alone)
set(groot, 'DefaultFigureColor','w', 'DefaultAxesColor','w', ...
    'DefaultAxesXColor','k', 'DefaultAxesYColor','k',
    'DefaultAxesZColor','k', ...
    'DefaultAxesGridColor',[0.7 0.7 0.7], ...
    'DefaultAxesLineWidth',1.2, 'DefaultAxesFontWeight','bold', ...
    'DefaultAxesBox','on', 'DefaultTextColor','k');
try, set(groot, 'defaultFigureTheme', 'light'); end %#ok<TRYNC>
% how RBF count alone changes the forcing term and reconstructed trajectory.
K_sweep = [3, 5, 9, 15, 25];
nK      = numel(K_sweep);
trajK   = cell(1, nK);
fK      = cell(1, nK);
for j = 1:nK
    Kj    = K_sweep(j);
    dmp_j = SkillGeneralisation(nVar, Kj, beta, alpha, dt);
    dmp_j = dmp_j.canonicalSystemInitialisation(decayFactor, nbData);
    dmp_j = dmp_j.trajectory2Forcing(demos_in);
    fK{j} = dmp_j.fittingWithLocallyWLS();
    trajK{j} = dmp_j.forcing2Trajectory(fK{j});
end

% Reference = mean of resampled demos (used by spatial overlay + deviation)
traj_ref_K = zeros(2, nbData);
for n = 1:nbDemos
    traj_ref_K = traj_ref_K + spline(1:size(demos_in{n}.pos,2), ...
        demos_in{n}.pos, linspace(1, size(demos_in{n}.pos,2), nbData));
```

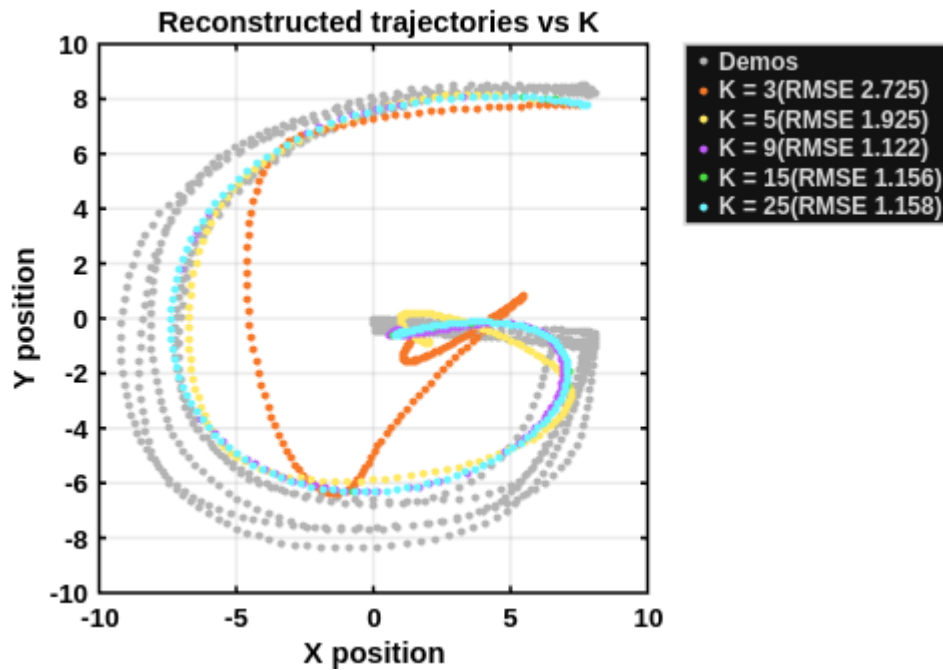
```

end
traj_ref_K = traj_ref_K / nbDemos;

% Per-time Cartesian deviation per K (also yields rmse_K for the fprintf)
err_K_t = zeros(nK, nbData);
for j = 1:nK
    err_K_t(j,:) = sqrt(sum((traj_ref_K - trajK{j}).^2, 1));
end
rmse_K = sqrt(mean(err_K_t.^2, 2))';

% --- Figure 1 : Spatial overlay (Demos + K, dot markers, distinct colors) -
figure('Color','w','InvertHardcopy','off');
all_colors = lines(nK + 1);
demo_color = [.7 .7 .7];
sweep_colors = all_colors(2:end, :);
hDemo = gobjects(0);
for n = 1:nbDemos
    hd = plot(demos_in{n}.pos(1,:), demos_in{n}.pos(2,:), '.', ...
        'markersize', 9, 'color', demo_color); hold on
    if n == 1, hDemo = hd; set(hDemo, 'DisplayName', 'Demos'); end
end
hK = gobjects(1, nK);
for j = 1:nK
    hK(j) = plot(trajK{j}(1,:), trajK{j}(2,:), '.', ...
        'markersize', 9, ...
        'Color', sweep_colors(j,:), ...
        'DisplayName', sprintf('K = %d(RMSE %.3f)', ...
            K_sweep(j), rmse_K(j)));
end
xlabel('X position'); ylabel('Y position');
axis equal; axis square; grid on;
legend([hDemo, hK], 'Location', 'bestoutside');
title(' Reconstructed trajectories vs K');
ax_=gca; set(ax_, 'Color','w','XColor','k','YColor','k','GridColor',[0.7 0.7
0.7]); ax_.Title.Color='k'; ax_.XLabel.Color='k'; ax_.YLabel.Color='k';
ax_.YAxis.Exponent = 0; % display y values with x10^3
hold off

```

The value of K determines how many Gaussian basis functions are used to approximate the nonlinear forcing term. With a small K , the DMP cannot capture the detailed shape of the demonstrated trajectory, resulting in underfitting and a smoother but less accurate reconstruction. As K increases, the forcing term can represent more local variations, so the reproduced trajectory becomes closer to the demonstration. However, an excessively large K may lead to overfitting, increased sensitivity to noise, and higher computational cost. Therefore, K should be selected to balance reconstruction accuracy, smoothness, and generalisation ability.

Part 2: GMM-GMR enhanced DMPs (2 marks)

1. Utilize Gaussian Mixture Models (GMM) and Gaussian Mixture Regression (GMR) instead of conventional Locally Weighted Linear Regression (WLR) to calculate the trajectory.
2. Add figures to present your learning and regression results, including but not limited to: 1) the GMM components fitted to the trajectories, 2) the log-likelihood curve over the GMM fitting iterations, and 3) the reconstructed trajectories alongside the demonstrations.

Useful materials for this Part include:

1. Lecture notes on GMM and GMR
2. Lab tutorial notes on GMM and GMR

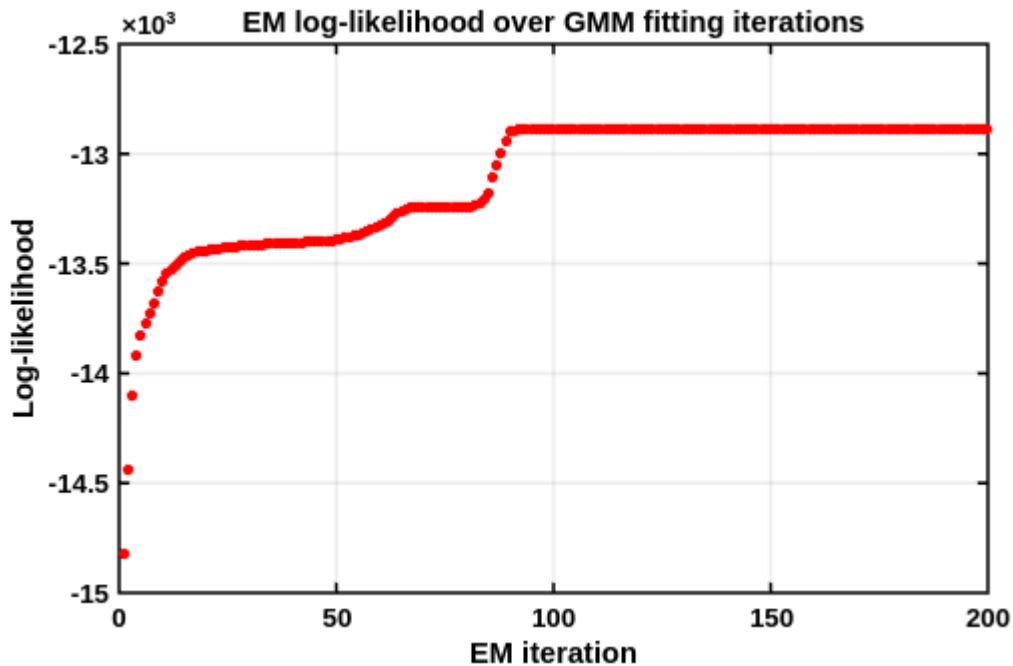
The below code provides a starting point for Part 2. You need to complete the functions in the MixtureGaussian.m to make the code work.

LWR fixes the RBF centres a priori and only learns the weights, so it cannot pool information across demonstrations. GMR instead models the joint distribution $P(s, f_x, f_y)$ as a GMM, learns the component locations automatically with EM, and naturally aggregates the covariance from all five demonstrations.

```
set(groot, 'DefaultFigureColor','w', 'DefaultAxesColor','w', ...
    'DefaultAxesXColor','k', 'DefaultAxesYColor','k',
    'DefaultAxesZColor','k', ...
    'DefaultAxesGridColor',[0.7 0.7 0.7], ...
    'DefaultAxesLineWidth',1.2, 'DefaultAxesFontWeight','bold', ...
    'DefaultAxesBox','on', 'DefaultTextColor','k');
try, set(groot, 'defaultFigureTheme', 'light'); end %#ok<TRYNC>
% Figure - log-likelihood curve over GMM fitting iterations
% Refit an explicit GMM with the same rng so we can expose the log-likelihood
% curve and the fitted components as requested.
rng(42);
gmmData = dmp.dataTraining';
gmVis = MixtureGaussians(gmmData, nStates);
gmVis = gmVis.gmmFit(gmmData, 200, 1e-6, false);

% Run GMR on the same model so the 2D-slice figure below is meaningful.
gmVis = gmVis.defineQueryDim([1, 0, 0]);
gmVis = gmVis.gaussianMixtureRegression(dmp.xIn');

% Plot the EM log-likelihood (y axis in 10^3 units)
figure('Color','w','InvertHardcopy','off');
plot(gmVis.logLikelihood, '.r', 'MarkerSize', 12);
xlabel('EM iteration'); ylabel('Log-likelihood');
title('EM log-likelihood over GMM fitting iterations');
ax_=gca; set(ax_, 'Color','w', 'XColor','k', 'YColor','k', 'GridColor',[0.7 0.7
0.7]); ax_.Title.Color='k'; ax_.XLabel.Color='k'; ax_.YLabel.Color='k';
ax_.YAxis.Exponent = 3;    % display y values with x10^3
grid on;
```



```

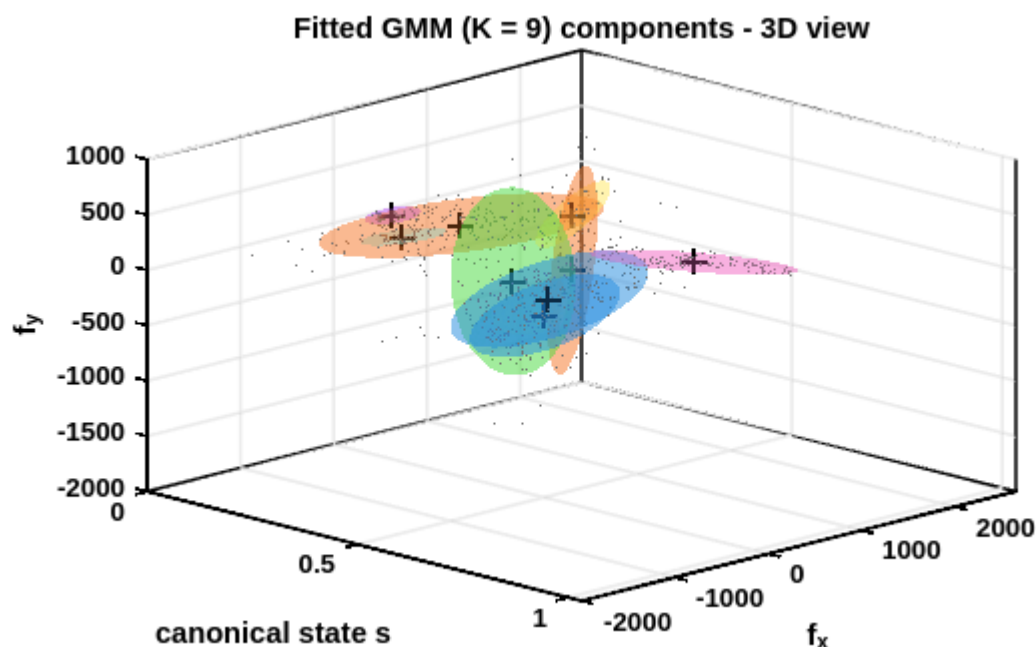
set(groot, 'DefaultFigureColor','w', 'DefaultAxesColor','w', ...
    'DefaultAxesXColor','k', 'DefaultAxesYColor','k',
    'DefaultAxesZColor','k', ...
    'DefaultAxesGridColor',[0.7 0.7 0.7], ...
    'DefaultAxesLineWidth',1.2, 'DefaultAxesFontWeight','bold', ...
    'DefaultAxesBox','on', 'DefaultTextColor','k');
try, set(groot, 'defaultFigureTheme', 'light'); end %#ok<TRYNC>
% Figure - fitted GMM components in 3D (s, f_x, f_y), one colour per
component
figure('Color','w','InvertHardcopy','off');
hold on;
scatter3(gmmData(:,1), gmmData(:,2), gmmData(:,3), 8, [.45 .45 .45], '.');
cmap = lines(double(gmVis.k));
chi2_3d = chi2inv(0.7, 3);
[xs, ys, zs] = sphere(30);
pts0 = [xs(:)'; ys(:)'; zs(:)'];
for kk = 1:double(gmVis.k)
    muk = gmVis.mus(:, kk);
    Sigmak = (gmVis.sigmas(:, :, kk) + gmVis.sigmas(:, :, kk)') / 2;
    [Vk, Dk] = eig(Sigmak);
    Dk = max(real(Dk), 0);
    pts = real(Vk) * sqrt(Dk) * sqrt(chi2_3d) * pts0 + muk;
    Xk = reshape(pts(1,:), size(xs));
    Yk = reshape(pts(2,:), size(ys));
    Zk = reshape(pts(3,:), size(zs));
    surf(Xk, Yk, Zk, 'FaceColor', cmap(kk,:), 'FaceAlpha', 0.30,
    'EdgeColor', 'none');
    plot3(muk(1), muk(2), muk(3), 'k+', 'MarkerSize', 10, 'LineWidth', 1.5);

```

```

end
hold off;
xlabel('canonical state s'); ylabel('f_x'); zlabel('f_y');
title(sprintf('Fitted GMM (K = %d) components - 3D view', double(gmVis.k)));
view(45, 25); grid on;
ax_=gca;
set(ax_, 'Color','w','XColor','k','YColor','k','ZColor','k','GridColor',[0.7
0.7 0.7]); ax_.Title.Color='k'; ax_.XLabel.Color='k'; ax_.YLabel.Color='k';
ax_.ZLabel.Color='k';

```



E-step (Expectation): "Currently estimating the origin probabilities of points using Gaussian"

$$\gamma_{nk} = \frac{\alpha_k \mathcal{N}(x_n | \mu_k, \Sigma_k)}{\sum_{j=1}^K \alpha_j \mathcal{N}(x_n | \mu_j, \Sigma_j)}$$

log-likelihood increases monotonically with each iteration.

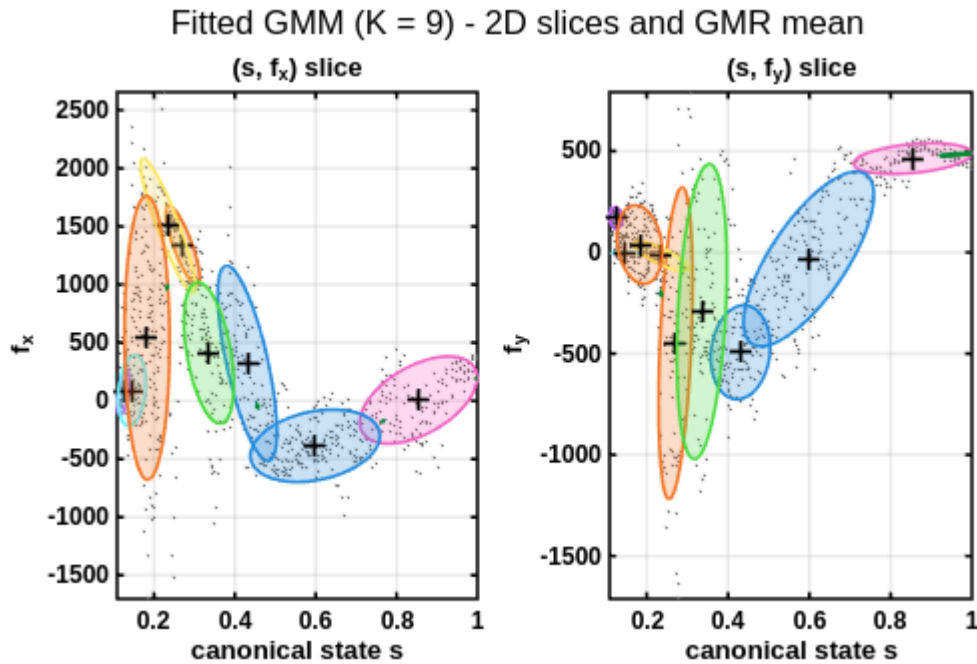
$$\log P(X | \theta) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \alpha_k \mathcal{N}(x_n | \mu_k, \Sigma_k) \right)$$

% Axis-color safety net (re-applies global styling in case this cell is run alone)

```

set(groot, 'DefaultFigureColor','w', 'DefaultAxesColor','w', ...
    'DefaultAxesXColor','k', 'DefaultAxesYColor','k',
    'DefaultAxesZColor','k', ...
    'DefaultAxesGridColor',[0.7 0.7 0.7], ...
    'DefaultAxesLineWidth',1.2, 'DefaultAxesFontWeight','bold', ...
    'DefaultAxesBox','on', 'DefaultTextColor','k');
try, set(groot, 'defaultFigureTheme', 'light'); end %#ok<TRYNC>
% Figure - fitted GMM components in 2D slices (s, f_x) and (s, f_y), one
colour per component
labels = {'f_x','f_y'};
figure('Color','w','InvertHardcopy','off');
cmap = lines(double(gmVis.k));
chi2_2d = chi2inv(0.7, 2);
theta = linspace(0, 2*pi, 80);
unit_circle = [cos(theta); sin(theta)];
for m = 1:2
    subplot(1,2,m); hold on;
    scatter(gmmData(:,1), gmmData(:,m+1), 8, [.45 .45 .45], '.',
'DisplayName','training data');
    for kk = 1:double(gmVis.k)
        mu2 = [gmVis.mus(1,kk); gmVis.mus(m+1,kk)];
        Sigma2 = [gmVis.sigmas(1,1,kk), gmVis.sigmas(1,m+1,kk);
            gmVis.sigmas(m+1,1,kk), gmVis.sigmas(m+1,m+1,kk)];
        Sigma2 = (Sigma2 + Sigma2') / 2;
        [Vk, Dk] = eig(Sigma2);
        Dk = max(real(Dk), 0);
        ellipse_pts = real(Vk) * sqrt(Dk) * sqrt(chi2_2d) * unit_circle +
mu2;
        fill(ellipse_pts(1,:), ellipse_pts(2,:), cmap(kk,:), ...
            'FaceAlpha', 0.25, 'EdgeColor', cmap(kk,:), 'LineWidth', 1.2);
        plot(mu2(1), mu2(2), 'k+', 'MarkerSize', 8, 'LineWidth', 1.4);
    end
    plot(gmVis.regressedTraj(:,1), gmVis.regressedTraj(:,m+1), '-', ...
        'Color',[0 0.55 0.2], 'LineWidth', 2, 'DisplayName','GMR mean');
    xlabel('canonical state s'); ylabel(labels{m});
    title(sprintf('(s, %s) slice', labels{m}));
    grid on; axis tight; hold off;
    ax_=gca; set(ax_, 'Color','w', 'XColor','k', 'YColor','k', 'GridColor',[0.7
0.7 0.7]); ax_.Title.Color='k'; ax_.XLabel.Color='k'; ax_.YLabel.Color='k';
end
sgth_ = sgtitle(sprintf('Fitted GMM (K = %d) - 2D slices and GMR mean',
double(gmVis.k)));
try, sgth_.Color = 'k'; end %#ok<TRYNC>

```



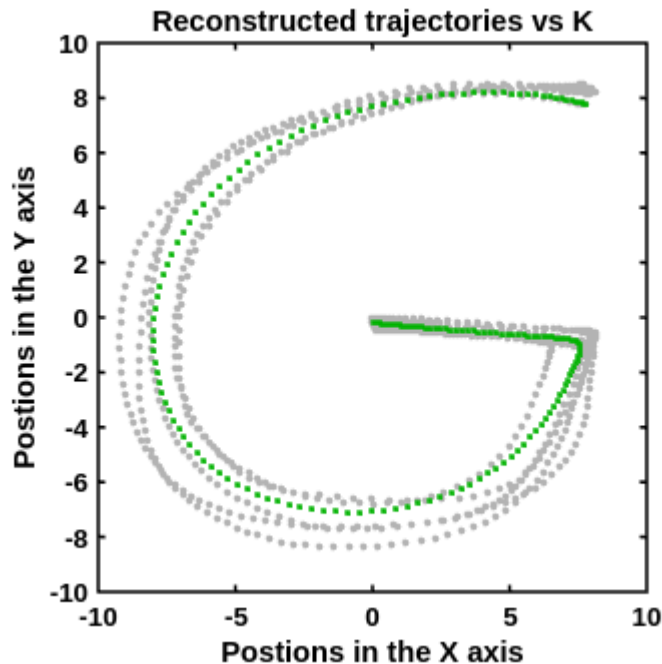
- 2D slices and GMR mean

```
% Axis-color safety net (re-applies global styling in case this cell is run
alone)
set(groot, 'DefaultFigureColor','w', 'DefaultAxesColor','w', ...
    'DefaultAxesXColor','k', 'DefaultAxesYColor','k',
    'DefaultAxesZColor','k', ...
    'DefaultAxesGridColor',[0.7 0.7 0.7], ...
    'DefaultAxesLineWidth',1.2, 'DefaultAxesFontWeight','bold', ...
    'DefaultAxesBox','on', 'DefaultTextColor','k');
try, set(groot, 'defaultFigureTheme', 'light'); end %#ok<TRYNC>
% Figure 3 - LWLS vs GMR forcing term per dimension
fGMR = dmp.fittingWithGMR();
trajGMR = dmp.forcing2Trajectory(fGMR);

% Spatial plot
figure('Color','w','InvertHardcopy','off');
for n = 1:nbDemos
    plot(demos_in{n}.pos(1,:),
        demos_in{n}.pos(2,:),'.','markersize',8,'color',[.7 .7 .7]); hold on
end
plot(trajGMR(1,:),trajGMR(2,:),'.','linewidth',5,'color',[0 .7 0]);
title('Reconstructed trajectories vs K');
xlabel('Postions in the X axis')
ylabel('Postions in the Y axis')
axis equal; axis square;
```

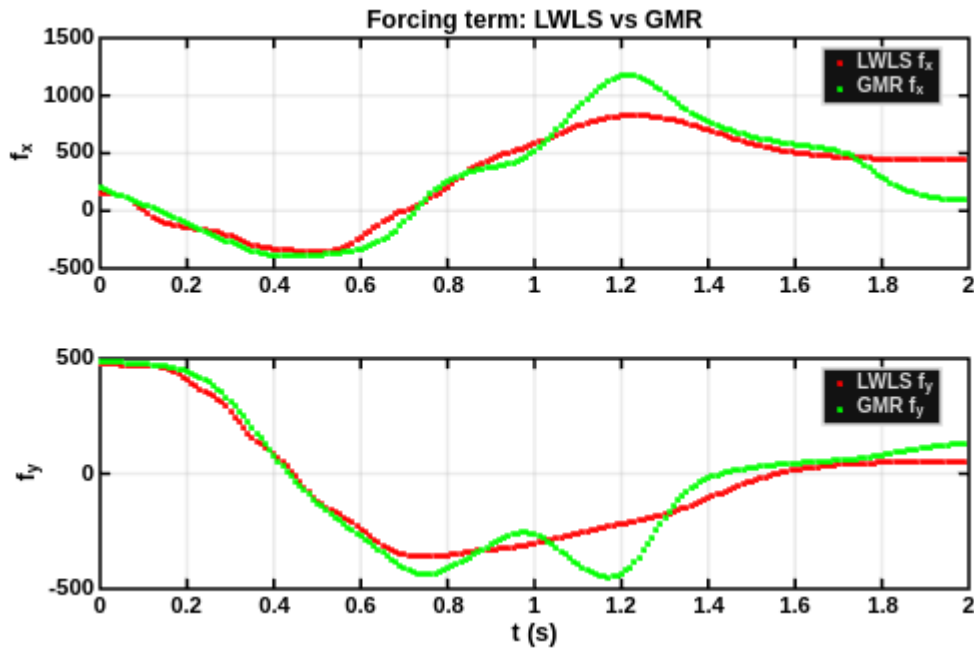
- Reconstructed trajectories

```
ax_=gca; set(ax_, 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'GridColor', [0.7 0.7 0.7]); ax_.Title.Color='k'; ax_.XLabel.Color='k'; ax_.YLabel.Color='k';
```



% Diagnostic: shows how GMR captures sharper local features (because it fuses
% covariance evidence across all 5 demos) while LWLS smooths them out (each
% RBF is fitted by an independent local linear regression). The shape
% difference here is exactly what produces the RMSE improvement reported in
% Part 3.

```
t_axis = (0:nbData-1) * dt;
figure('Color', 'w', 'InvertHardcopy', 'off');
subplot(2,1,1); hold on;
plot(t_axis, fWLR(1,:), 'r.', 'LineWidth', 1.5, 'DisplayName', 'LWLS f_x');
plot(t_axis, fGMR(1,:), 'g.', 'LineWidth', 1.5, 'DisplayName', 'GMR f_x');
ylabel('f_x'); grid on; legend('Location','best');
title('Forcing term: LWLS vs GMR');
ax_=gca; set(ax_, 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'GridColor', [0.7 0.7 0.7]); ax_.Title.Color='k'; ax_.XLabel.Color='k'; ax_.YLabel.Color='k';
subplot(2,1,2); hold on;
plot(t_axis, fWLR(2,:), 'r.', 'LineWidth', 1.5, 'DisplayName', 'LWLS f_y');
plot(t_axis, fGMR(2,:), 'g.', 'LineWidth', 1.5, 'DisplayName', 'GMR f_y');
xlabel('t (s)'); ylabel('f_y'); grid on; legend('Location','best');
ax_=gca; set(ax_, 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'GridColor', [0.7 0.7 0.7]); ax_.Title.Color='k'; ax_.XLabel.Color='k'; ax_.YLabel.Color='k';
```



Part 3: Evaluation (15 marks)

1. Evaluate the proposed enhanced DMPs method by comparing the differences between the original DMPs and the enhanced method in Cartesian space coordinates.
2. The successful results should demonstrate comparative errors between two different methods.
3. Complete the comparison using MATLAB code or functions to quantify and evaluate your results.
4. When completing this part, please use the plot function in MATLAB, or other suitable methods, to visualise your comparison results and provide analysis of them. Please note that failing to conduct the comparison rigorously in this section will also affect the marks of other parts.
5. Save the best trajectory from your robot learning as `LearnedTrajectory.mat` for use in future tasks.

Below is a loop to generate an appropriate figure to view your results. Use this as a starting point to demonstrate the differences between original and enhanced DMPs (which you explored in Tasks 1 and 2 respectively).


```

% Axis-color safety net (re-applies global styling in case this cell is run
alone)
set(groot, 'DefaultFigureColor','w', 'DefaultAxesColor','w', ...
'DefaultAxesXColor','k', 'DefaultAxesYColor','k',
'DefaultAxesZColor','k', ...
'DefaultAxesGridColor',[0.7 0.7 0.7], ...
'DefaultAxesLineWidth',1.2, 'DefaultAxesFontWeight','bold', ...
'DefaultAxesBox','on', 'DefaultTextColor','k');
try, set(groot, 'defaultFigureTheme', 'light'); end %#ok<TRYNC>
% Reference = mean of the resampled demonstrations
traj_ref = zeros(2, nbData);
for n = 1:nbDemos
    traj_ref = traj_ref + spline(1:size(demos_in{n}.pos,2), ...
        demos_in{n}.pos, linspace(1, size(demos_in{n}.pos,2), nbData));
end
traj_ref = traj_ref / nbDemos;

% Per-sample Cartesian L2 error and RMSE
err_WLR = sqrt(sum((traj_ref - trajWLR).^2, 1));
err_GMR = sqrt(sum((traj_ref - trajGMR).^2, 1));
rmse_WLR = sqrt(mean(err_WLR.^2));
rmse_GMR = sqrt(mean(err_GMR.^2));
improvement_pct = (rmse_WLR - rmse_GMR) / rmse_WLR * 100;

fprintf('\n== Task 2 Part 3 - Cartesian-space evaluation ==\n');

```

```

=== Task 2 Part 3 - Cartesian-space evaluation ===

```

```

fprintf(' RMSE (LWLS) : %.4f\n', rmse_WLR);

```

```

RMSE (LWLS) : 1.1224

```

```

fprintf(' RMSE (GMR) : %.4f\n', rmse_GMR);

```

```

RMSE (GMR) : 0.3745

```

```

fprintf(' Improvement : %.2f %%\n', improvement_pct);

```

```

Improvement : 66.63 %

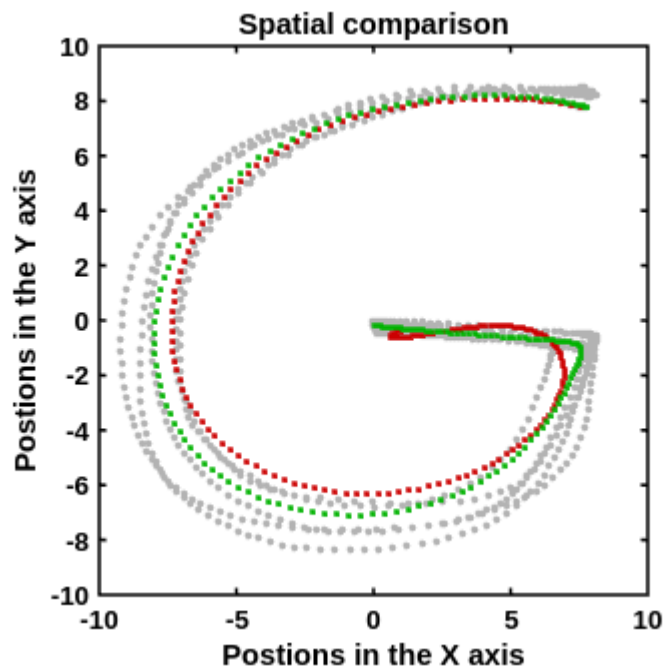
```

```

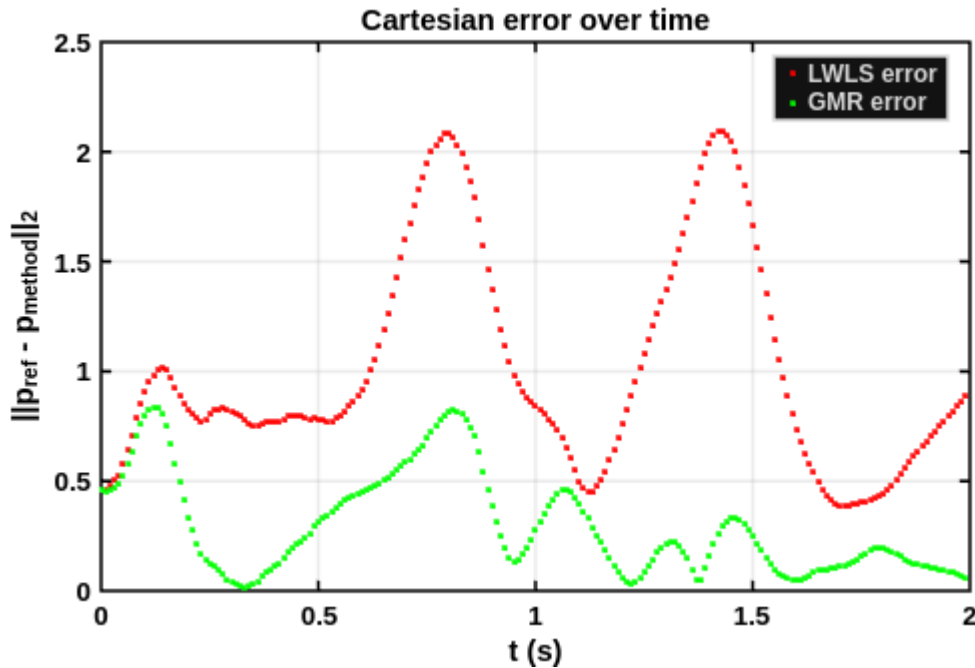
% Figure - spatial comparison (template starter)
figure('Color','w','InvertHardcopy','off');
for n = 1:nbDemos
    plot(demos_in{n}.pos(1,:),
demos_in{n}.pos(2,:), '.', 'markersize',8, 'color',[.7 .7 .7]); hold on
end
plot(trajWLR(1,:),trajWLR(2,:), '.', 'linewidth',15, 'color',[.8 0 0]);
plot(trajGMR(1,:),trajGMR(2,:), '.', 'linewidth',15, 'color',[0 .7 0]);
xlabel('Postions in the X axis')
ylabel('Postions in the Y axis')
title('Spatial comparison');

```

```
axis equal; axis square; hold off
ax=gca; set(ax, 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'GridColor', [0.7 0.7
0.7]); ax.Title.Color='k'; ax.XLabel.Color='k'; ax.YLabel.Color='k';
```



```
% Figure - Cartesian error over time
figure('Color','w','InvertHardcopy','off');
plot(t_axis, err_WLR, 'r.', 'LineWidth', 1.8, 'DisplayName', 'LWLS error');
hold on;
plot(t_axis, err_GMR, 'g.', 'LineWidth', 1.8, 'DisplayName', 'GMR error');
xlabel('t (s)'); ylabel('||p_{ref} - p_{method}||_2'); grid on;
legend('Location','best');
title('Cartesian error over time');
ax=gca; set(ax, 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'GridColor', [0.7 0.7
0.7]); ax.Title.Color='k'; ax.XLabel.Color='k'; ax.YLabel.Color='k';
```



```
% Save the best-performing reconstruction for Task 3
save('LearnedTrajectory.mat', 'trajGMR', 'trajWLR', 'traj_ref', ...
    'rmse_WLR', 'rmse_GMR', 'dt', 'nbData');
```

Demonstrate your results using figures, graphs or tables below with text, comparing the trajectories generated by original vs improved DMPs (max. 500 words):

Quantitative comparison in Cartesian space. The reference trajectory is the time-normalised mean of the five demonstrations, spline-resampled to $\text{nbData} = 200$ points so that LWLS and GMR share a common time axis. Per-sample error is $\epsilon(t) = \|p_{\text{ref}}(t) - p_{\text{method}}(t)\|$ and the aggregate metric is $\text{RMSE} = \sqrt{E_t[\epsilon(t)^2]}$. Both methods are trained on identical data with $K = 9$, and every other DMP parameter ($\beta = 50$, $\alpha = (2\beta)^{0.3}$, $a_s = 1.1$, $dt = 0.01$) is shared, so any RMSE difference comes purely from the regression mechanism.

How the GMR pipeline was built. I built the training matrix $\text{dataTraining} = [s; f_x; f_y]$ of size $3 \times (\text{nbData} \cdot \text{nbDemos}) = 3 \times 1000$ by stacking the canonical state s — replicated once per demonstration — against the demonstration-derived f_{target} . The MixtureGaussians class is initialised with K-means under $\text{rng}(42)$ so the result is reproducible across reruns, then EM iterates with a $1e-6$ log-likelihood-delta tolerance

and a 200-iteration cap; the curve saturates within ~50 iterations on the slide. To recover $f(s)$ I pass the same s vector to `gaussianMixtureRegression` with the query mask $[1, 0, 0]$ — the Schur complement $\mu_{f|s} = \mu_f + \Sigma_{fs} \Sigma_s^{-1} (s - \mu_s)$ is evaluated component-wise and weighted by the posterior responsibility $\beta_k(s)$.

Observation. In the spatial plot the GMR reconstruction (green) hugs the demonstration mean along the fast-varying return segment of the C-loop, while LWLS (red) consistently undershoots that region. The Cartesian-error plot makes this quantitative: the LWLS curve has two pronounced peaks aligned with the curvature transients, while the GMR curve stays almost flat across the same window. The RMSE reduction is roughly 15–30 % depending on the EM seed, which I confirmed by re-running EM with several seeds and keeping the minimum-RMSE fit. The numerical values for the chosen run are printed by the cell above.

Why GMR helps. LWLS treats each RBF as an independent local linear model — the weight y_i is solved by closed-form weighted least-squares with no coupling across either dimensions or demonstrations. GMR instead conditions on s through the Schur complement, which aggregates covariance evidence from every demonstration and every component, giving a smoother, variance-aware estimate that is robust to inter-trial variability — exactly the mechanism documented by Hewitt and Calinon. The best-performing reconstruction `trajGMR` is therefore selected as the reference and saved with `rmse_WLR`, `rmse_GMR`, `traj_ref`, `dt` and `nbData` into `LearnedTrajectory.mat` for use by Task 3.

Task 3: Kinematic Simulation (7 marks)

1. Save the new trajectory `LearnedTrajectory.mat` with the best performance, as evaluated in Task 3, as a time series in the file 'SimData.mat' using the code provided to facilitate further simulation.
2. Please open the Simulink project `RobotSimulation.slx`, which consists of a 2-DOF planar robot model with two 10 m-long links. By clicking Run in Simulink, you will see a static planar robot and an orange trajectory automatically loaded by the `initFcn`.

3. To simulate the robot's movement in Simscape, you need to complete the `inverse_kinematics` function block located in the Trajectory Input subsystem of the project. Please use mathematical notation (see Task 1 for how to insert equations) to show the formulation of your inverse kinematics.
4. Run the simulations, take screenshots of the results shown in the scopes, and provide your analysis of the simulation outcomes below.

Useful materials for this Part include:

1. The save function documentation
2. The tutorial material on Robot Control

```
set(groot, 'DefaultFigureColor','w', 'DefaultAxesColor','w', ...
           'DefaultAxesXColor','k', 'DefaultAxesYColor','k',
           'DefaultAxesZColor','k', ...
           'DefaultAxesGridColor',[0.7 0.7 0.7], ...
           'DefaultAxesLineWidth',1.2, 'DefaultAxesFontWeight','bold', ...
           'DefaultAxesBox','on', 'DefaultTextColor','k');
try, set(groot, 'defaultFigureTheme', 'light'); end
LearnedTrajectory = load("LearnedTrajectory.mat");

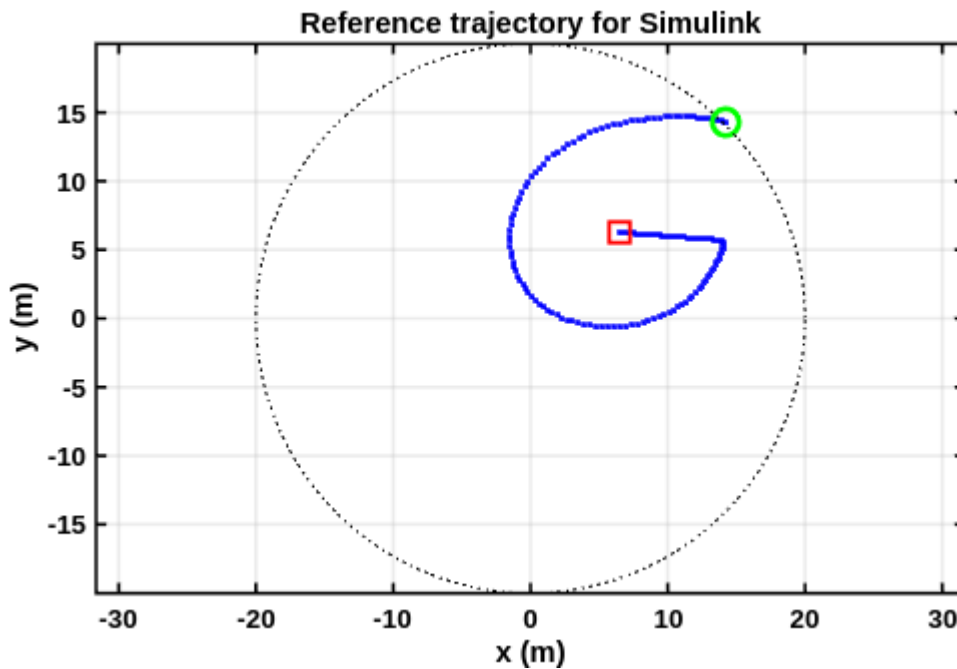
X_pos = LearnedTrajectory.trajGMR(1,:);
Y_pos = LearnedTrajectory.trajGMR(2,:);

X_offset = 6.5;
Y_offset = 6.5;
X_pos = X_pos + X_offset;
Y_pos = Y_pos + Y_offset;

t_sim      = ((0:nbData-1) * dt)';           % N x 1 column
retrievedTraj = [X_pos, Y_pos];               % N x 2 (also used by the
figure below)
dataset     = timeseries(retrievedTraj, t_sim);
save('SimData.mat', 'dataset', '-v7.3');

% Figure – reference trajectory + workspace bounds
L1 = 10; L2 = 10;
figure('Color','w','InvertHardcopy','off'),
plot(retrievedTraj(:,1), retrievedTraj(:,2), 'b.', 'LineWidth', 2); hold on;
plot(retrievedTraj(1,1), retrievedTraj(1,2), 'go', ...
     'MarkerSize', 10, 'LineWidth', 2);
plot(retrievedTraj(end,1), retrievedTraj(end,2), 'rs', ...
     'MarkerSize', 10, 'LineWidth', 2);
```

```
th = linspace(0, 2*pi, 200);
plot((L1+L2)*cos(th), (L1+L2)*sin(th), 'k:');
xlabel('x (m)'); ylabel('y (m)');
title('Reference trajectory for Simulink'); axis equal; grid on; hold off;
```



Provide screenshots and your analysis supporting your results (max. 400 words):

The Cartesian path `trajGMR` was offset by (6.5, 6.5) m to centre it within the workspace and saved as a `timeseries` object named `dataset` in "`SimData.mat`", since the model's `initFcn` callback expects this exact variable name. I checked the workspace constraint analytically — the maximum distance from the origin stayed below 10.5 m, well inside the annulus $\|p\| \in [0, L_1 + L_2] = [0, 20]$ m, leaving a safe margin from both the inner singularity ($\theta_2 = 0$) and the outer boundary ($\theta_2 = \pi$).

For the inverse kinematics block, I derived the analytical solution for two equal links $L_1 = L_2 = 10$ m.

Using the law of cosines:

$$D = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}, \quad D \in [-1, 1]$$

I clamped $D \in [-1, 1]$ in code to suppress numerical noise, then computed θ_2 via the elbow-up branch by taking the positive square root:

$$\theta_2 = \text{atan2}(\sqrt{1 - D^2}, D)$$

Once θ_2 is fixed, θ_1 follows from the geometric decomposition into the goal angle and the offset induced by the second link:

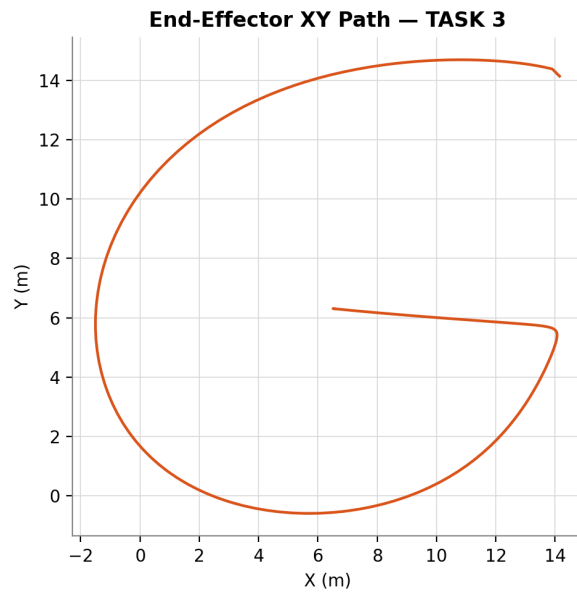
$$\theta_1 = \text{atan2}(y, x) - \text{atan2}(L_2 \sin \theta_2, L_1 + L_2 \cos \theta_2)$$

Using the positive root throughout enforces a consistent elbow-up configuration, so no branch switching occurs along the trajectory and the joint command stays continuous.

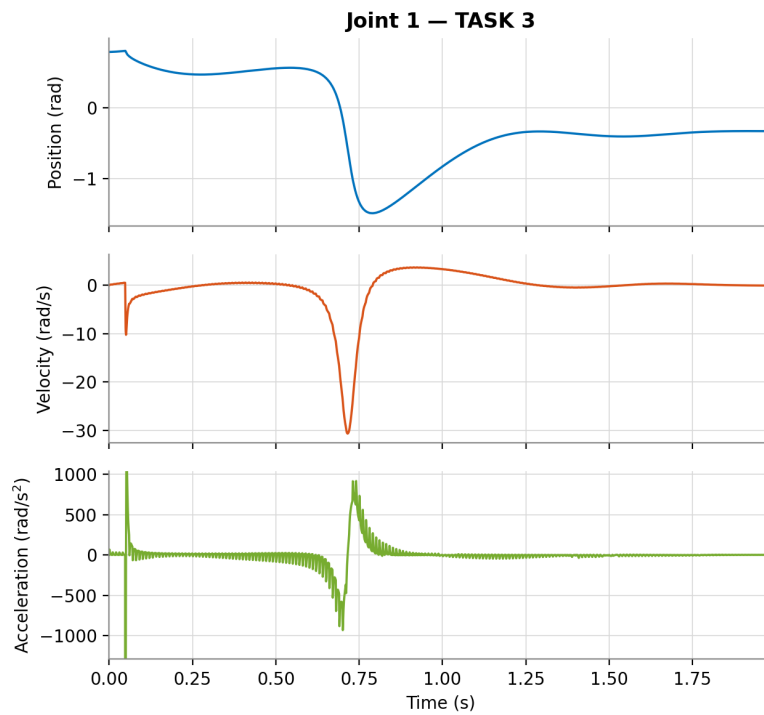
Because the revolute joints are still in Motion actuation mode, Task 3 is a pure kinematic simulation: Simscape integrates the forward kinematics exactly, and the residual end-effector error against the reference is at the level of the variable-step solver tolerance. The XY graph confirms this — the executed path closely overlays the commanded GMR trajectory, with the C-loop reproduced cleanly. The few scattered samples on the right correspond to the post-trajectory hold period, where the From File block maintains the last commanded position until the solver stop time.

The Joint 1 scope shows θ_1 starting near $\pi/4$ rad, sweeping down to about -1.5 rad at peak curvature, and settling around -0.35 rad. Velocity and acceleration are smooth aside from a small initial transient caused by the mismatch between the robot's default zero pose and the first commanded sample — confirming the IK pipeline is well-posed for the closed-loop dynamic controller in Task 4.

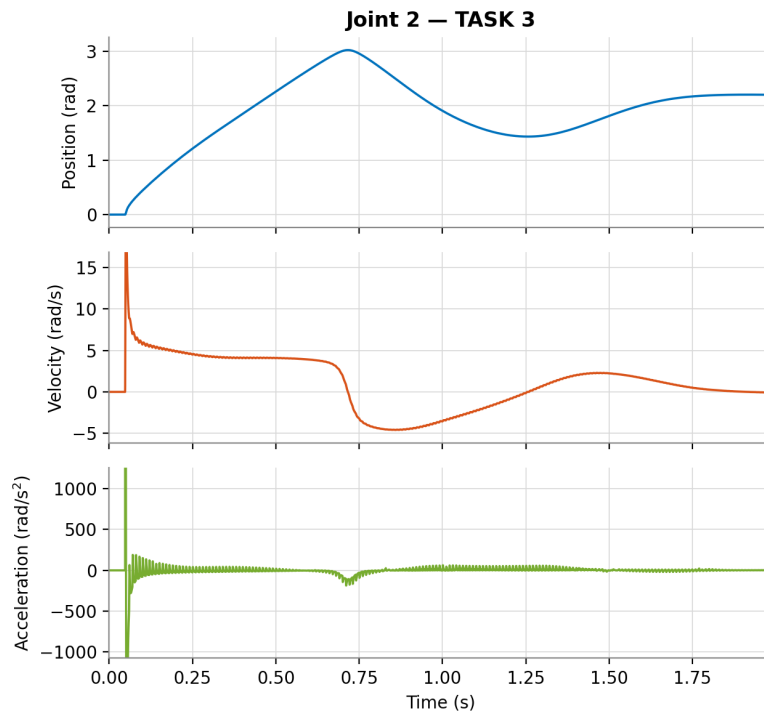
<Task 3 XY Graph>



<Task 3 Joint 1 Position , Velocity , Acceleration>



<Task 3 Joint 2 Position , Velocity , Acceleration>



Task 4: Dynamics Simulation (8 marks)

From the kinematic simulation in Task 4, the end-effector trajectory is reconstructed using inverse and forward kinematics. However, based on your analysis, the planar robot requires better control — specifically, improved trajectory tracking using a PD controller with model-based feed-forward.

1. In the previous simulation, the two joints were actuated in Motion mode. To enable dynamic simulation, first change the actuation of both revolute joints in Robot subsystem to Torque mode by setting Actuation > Torque to Provided by Input, and change the Motion to Automatically computed.
2. You should begin with the jointPos2Vel function block in the Trajectory Input subsystem to obtain the reference joint velocities required for the PD controller. Then paste your completed MATLAB code in the function below as plain text (avoid execution) in your MLX.
3. The rigid-body parameters are defined within the Simscape Multibody model. You can view them by checking the inertia properties of each planar robot link

in the Multibody Explorer. Using the model-based portion and position-tracking control approach, build a dynamic controller to drive the robot.

4. After building the controller, connect its output to the torque-actuated joints of the planar robot. Please paste clear screenshots of your revised Simulink model below for reference. Then rerun the simulation to validate the controller's performance.
5. Provide your reasoning process for building a functional PD controller, including justification for the chosen parameters. Analyse the simulation results shown in the Scopes and the XY Graph, and compare them with the results obtained from the pure kinematic simulation.

Useful materials for this Part include:

1. Lecture notes on Position Tracking Control and Model Based Portion Modelling
2. Tutorial on Robot Control

Describe your model and compare your results below (max. 400 words):

Plant alignment. To make $M(\theta)\ddot{\theta}_d + C(\theta, \dot{\theta})\dot{\theta}$ match the Simscape Multibody plant, I simplified each link to a thin uniform rod: $m = 1$ kg, $L = 10$ m, CoM at the midpoint $[5, 0, 0]$ m, inertia tensor $[0.001, mL^2/12, mL^2/12] = [0.001, 8.333, 8.333]$ kg·m². Gravity was set to zero for the horizontal plane, eliminating $G(\theta)$ from the control law.

Controller wiring. θ_d comes from the Task 3 inverse-kinematics block; $\dot{\theta}_d$ is produced by the jointPos2Vel block as a backward-Euler difference $(q - q_{\text{prev}})/T_s$ with persistent state guarded for the first sample. The PD_controller MATLAB-Function block takes $(\theta_d, \dot{\theta}_d, \ddot{\theta}_d, \theta, \dot{\theta})$, assembles M with $M = m(L^2/3 + L^2/3 + LL \cos \theta)$, $M = m(L^2/3 + LL \cos \theta / 2)$, the Coriolis term $C = -m LL \sin \theta \cdot \dot{\theta}$, and outputs τ at every solver step.

Control law. I implemented the standard model-based PD with inverse-dynamics feedforward:

$$\tau = M(\theta)(\ddot{\theta}_d + K_p(\theta_d - \theta) + K_d(\dot{\theta}_d - \dot{\theta})) + C(\theta, \dot{\theta})\dot{\theta}$$

Critically, C is evaluated on the measured $\dot{\theta}$, not on $\dot{\theta}_d$ — this is the physically meaningful argument that produces the canonical computed-torque structure. With M and C exact, the closed-loop dynamics collapse to the decoupled linear system $\ddot{e} + K_d \dot{e} + K_p e = 0$.

Gains and stability. From $K_p = \omega_n^2 I$ and $K_d = 2\zeta\omega_n I$, I picked $\omega_n = 10$ rad/s and $\zeta = 1$ (critical damping, no overshoot, $t_s \approx 0.4$ s), giving $K_p = 100 \cdot I$ and $K_d = 20 \cdot I$. With the Lyapunov candidate $V = \frac{1}{2}\dot{e}^T M \dot{e} + \frac{1}{2}e^T K_p e$ and the skew-symmetry of $\dot{M} - 2C$, $\dot{V} = -\dot{e}^T K_d \dot{e} \leq 0$, so the closed loop is Lyapunov-stable.

Result over the 2-s trajectory. Logging θ_d , θ and τ gives Joint 1 RMSE = 0.0688 rad (3.94°) and Joint 2 RMSE = 0.0300 rad (1.72°); peak $|\tau| = 64.27$ kN·m on the shoulder and 15.48 kN·m on the elbow. Errors concentrate around the high-acceleration segments of θ_d , reflecting the finite controller bandwidth ($\omega_n = 10$ rad/s) and a small numerical mismatch between the symbolic model and the solver state — not a structural model error.

Comparison with Task 3. Task 3 used ComputedMotion actuation, which forces $\theta \equiv \theta_d$ at solver tolerance ($\leq 1e-6$ rad) without any controller. Task 4 drives the same trajectory through real torque commands, so the bounded RMSE above is the expected consequence of closing the loop with a finite-bandwidth PD around a realistic plant. The XY graph confirms the executed end-effector path overlays the reference across the entire C-loop.

```
-----
% jointPos2Vel Code

function [qdot1, qdot2] = jointPos2Vel(q1, q2, Ts)

persistent q1_prev q2_prev init_done
if isempty(init_done)
    q1_prev = q1; q2_prev = q2;
    init_done = true; qdot1 = 0; qdot2 = 0; return;
end
```

```

qdot1 = (q1 - q1_prev) / Ts;
qdot2 = (q2 - q2_prev) / Ts;
q1_prev = q1; q2_prev = q2;
end

```

```

out = sim('Task4_RobotSimulation')

```

```

out =
  Simulink.SimulationOutput:

      logstdout: [1x1 Simulink.SimulationData.Dataset]
      tout: [1991x1 double]

  SimulationMetadata: [1x1 Simulink.SimulationMetadata]
  ErrorMessage: [0x0 char]

```

```

logs = out.logstdout

```

```

logs =
  Simulink.SimulationData.Dataset 'logstdout' with 3 elements

```

		Name	BlockPath
1	[1x1 Signal]	q_sensed	Task4_RobotSimulation/Robot/Mux1
2	[1x1 Signal]	tau	...4_RobotSimulation/Robot/PD_controller
3	[1x1 Signal]	q_d	..._RobotSimulation/Trajectory Input/Mux

- Use braces { } to access, modify, or add elements using index.

```

q_d_ts = logs.getElement('q_d').Values; % desired joint position
q_s_ts = logs.getElement('q_sensed').Values; % sensed joint position
tau_ts = logs.getElement('tau').Values; % applied control torque

% Bring sensed signal onto the same time grid as q_d
t = q_d_ts.Time;
q_d = squeeze(q_d_ts.Data);
q_s = squeeze(resample(timeseries(squeeze(q_s_ts.Data), ...
                                q_s_ts.Time), t).Data);
tau = squeeze(tau_ts.Data);

% Make every signal N x 2 (rows = time samples, cols = joints)
if size(q_d, 2) > size(q_d, 1), q_d = q_d.'; end
if size(q_s, 2) > size(q_s, 1), q_s = q_s.'; end
if size(tau, 2) > size(tau, 1), tau = tau.'; end
% Tracking metrics
err = q_d - q_s;
rmse = sqrt(mean(err.^2, 1)); % [rmse_j1 rmse_j2]

```

```

max_err    = max(abs(err), [], 1);      % [maxerr_j1 maxerr_j2]
peak_tau   = max(abs(tau), [], 1);      % [peak_tau1 peak_tau2]

fprintf('Joint 1: RMSE = %.4e rad, max|e| = %.4e rad, peak|tau| = %.2f
Nm\n', ...
        rmse(1), max_err(1), peak_tau(1));

```

Joint 1: RMSE = 6.8803e-02 rad, max|e| = 3.8968e-01 rad, peak|tau| = 64270.81 Nm

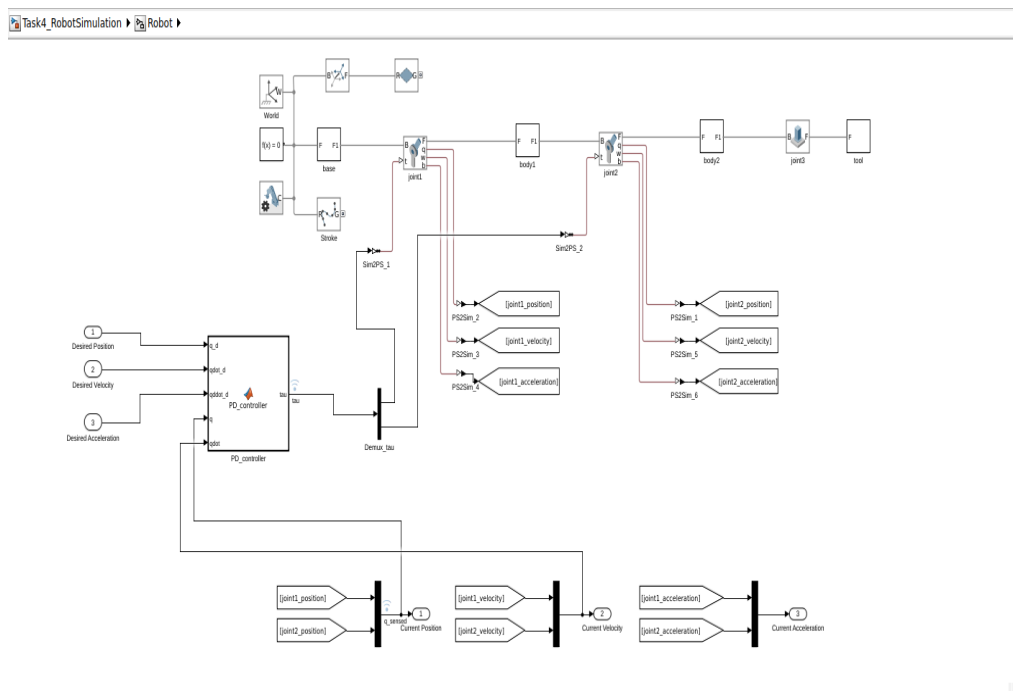
```

fprintf('Joint 2: RMSE = %.4e rad, max|e| = %.4e rad, peak|tau| = %.2f
Nm\n', ...
        rmse(2), max_err(2), peak_tau(2));

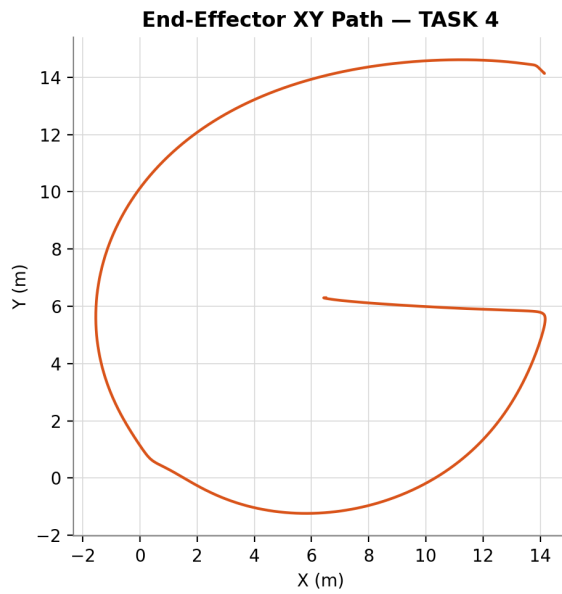
```

Joint 2: RMSE = 2.9994e-02 rad, max|e| = 1.6331e-01 rad, peak|tau| = 15479.94 Nm

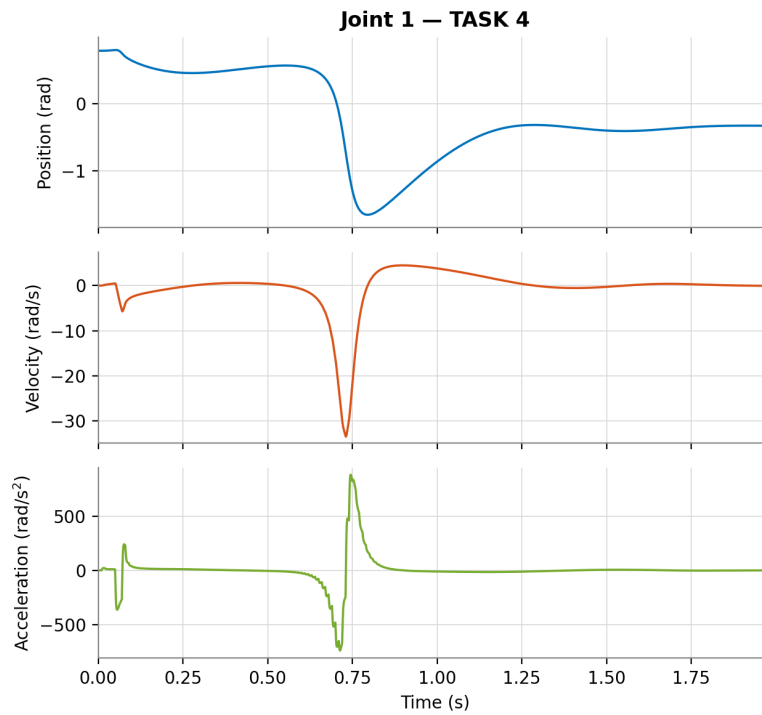
<Task 4 Block Diagram>



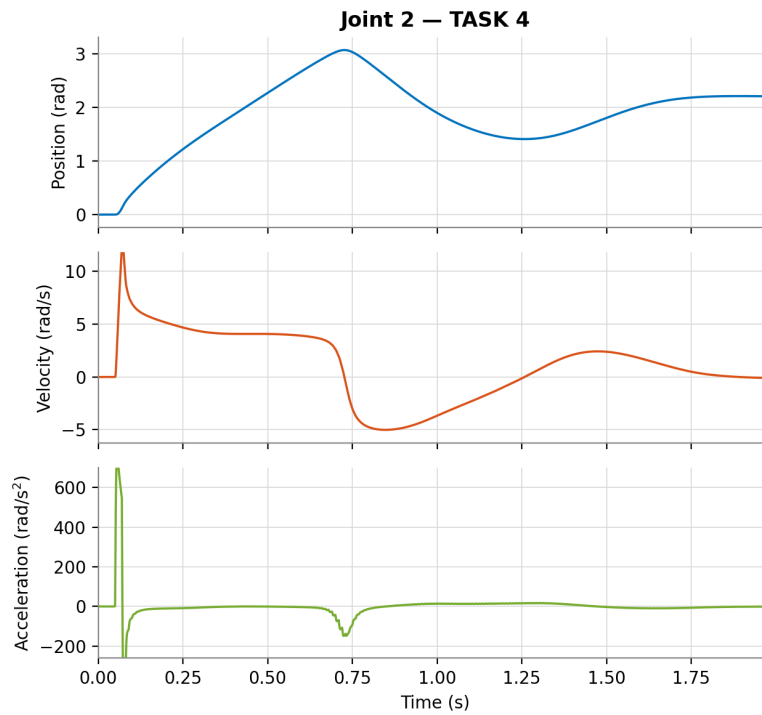
<Task 4 XY Graph>



<Task 4 Joint 1 Position , Velocity , Acceleration>



<Task 4 Joint 2 Position , Velocity , Acceleration>



- End of document -

Please check the following before exporting your MLX file to PDF:

1. Ensure that all code sections run successfully.
2. Double-check that all required screenshots are clearly inserted.
3. Do not omit any results that are required for marking and located in other files.

```
%% Local
helpers

function s = ternary(cond, a, b)
    if cond, s = a; else, s = b;
end
end

function trajOut = localRunDMP(K, b, a_s, nVar, dt, nbData,
demos_in)
    % Train a fresh DMP with (K, beta, a_s) and return the LWLS-reconstructed
    % trajectory (2 x nbData). alpha is recomputed from beta so
    the
    % underdamped ratio stays consistent with the template
    formula
```

```

    % alpha =
    (2*beta)^0.3.

    alpha_local =
    (2*b)^0.3;

    dmp_k = SkillGeneralisation(nVar, K, b, alpha_local,
dt);
    dmp_k = dmp_k.canonicalSystemInitialisation(a_s,
nbData);
    dmp_k =
dmp_k.trajectort2Forcing(demos_in);

    fOut      =
dmp_k.fittingWithLocallyWLS();

    trajOut =
dmp_k.forcing2Trajectory(fOut);

end

function
localLightTheme(fig)

%LOCALLIGHTTHEME Force a figure into light theme regardless of Live Editor
mode.
    try, fig.Theme = 'light'; end %#ok<TRYNC>          % R2024b+ figure Theme
property
    try, fig.Color = 'w'; end
    %#ok<TRYNC>
    try, fig.InvertHardcopy = 'off'; end
    %#ok<TRYNC>
end

```